educación IT
CODE YOUR FUTURE

Solución de errores con inteligencia artificial

Depuración asistida por IA

La depuración asistida por IA **combina las técnicas tradicionales de depuración con el poder analítico de la inteligencia artificial**, lo que representa una evolución significativa en la forma en que identificamos y resolvemos errores en nuestro código Python.

Cuando utilizamos herramientas de IA para depurar, estamos aprovechando **modelos entrenados con millones de líneas de código y patrones de error comunes**.

Estas herramientas pueden analizar el código, **identificar potenciales problemas** antes de que se manifiesten y **sugerir soluciones** basadas en las mejores prácticas de la comunidad de desarrolladores.



Los asistentes de IA como GitHub Copilot o Chat GPT **ofrecen explicaciones detalladas de los errores** en lenguaje natural.

Una característica particularmente útil es el **análisis de patrones de error históricos**. Las IA pueden identificar similitudes entre el error actual y problemas anteriores en nuestro código o en repositorios públicos similares, y sugieren soluciones que han funcionado en casos parecidos.

En cuanto a la **optimización de código**, las herramientas de IA pueden analizar nuestras estructuras de datos y algoritmos, y luego proponer alternativas más eficientes.



Es fundamental recordar que la IA actúa como un asistente y no como un reemplazo de nuestro criterio como programadores. **Debemos evaluar críticamente cada sugerencia,** comprender por qué la solución propuesta resuelve el problema (o no) y asegurarnos de que se alinee con nuestros objetivos.



Usar *prompts* adecuados

La **formulación adecuada de *prompts*** es necesaria para obtener soluciones precisas. Al trabajar con una IA, debemos **proporcionar el contexto completo del problema**:

- **El código que genera el error**: No basta con compartir solo la línea que genera el error; debemos incluir el contexto suficiente. Por ejemplo, si tenemos un error en una función, debemos mostrar la definición completa de la función, las variables que utiliza y cómo se llama.
- **El mensaje de error exacto**: Incluir el *traceback* completo. Por ejemplo, en lugar de decir “tengo un IndexError”, debemos compartir la traza completa que nos muestra Python, incluyendo el número de línea y la secuencia de llamadas que llevó al error.
- **El comportamiento esperado**: Debemos describir claramente qué intentamos lograr con nuestro código. Por ejemplo, podemos explicar “estoy intentando filtrar una lista de números para obtener solo los pares, pero mi código genera un error cuando la lista está vacía”.

Ejemplos de *prompts* deficientes

"Hola, mi código no funciona. Me sale error con la siguiente operación. Ayuda por favor!!!"

"Tengo este código:

```
num1 = "2"
resultado = num1 + num2
```

No sirve, ¿qué hago?"



Estos *prompts* son problemáticos por múltiples razones. Primero, no proporcionan el código que genera el error. Segundo, las frases "no funciona" o "no sirve" son demasiado vagas y no describen el comportamiento actual ni el deseado.

Tercero, "me sale error" no especifica qué tipo de error está ocurriendo. Cuarto, omiten información crucial para el diagnóstico: no sabemos si `num2` está definida, si se intenta sumar o concatenar.

Este tipo de *prompts* generalmente **resulta en respuestas genéricas o en una serie de preguntas de seguimiento** por parte de la IA, lo que hace el proceso de depuración más largo y menos eficiente.



Ejemplo de un buen *prompt*

"Estoy trabajando con Python 3.9 y tengo el siguiente código que debería calcular el promedio de valores ingresados por el usuario:

```
num1 = input("Ingrese el primer valor: ")
num2 = input("Ingrese el segundo valor: ")
num3 = input("Ingrese el tercer valor: ")
resultado = (num1 + num2 + num3) / 3
```

Recibo este error:

```
TypeError: unsupported operand type(s) for /: 'str' and 'int'
```

Deseo obtener un valor numérico.
¿Cómo puedo modificar mi código?"



El *prompt* es efectivo porque **incluye el código completo, el error exacto, el comportamiento actual y el comportamiento deseado**. La IA podrá entender claramente el problema y proporcionar una solución específica.

También es importante **mencionar cualquier restricción o requerimiento específico**. Por ejemplo, si necesitamos mantener cierta estructura de código, usar ciertas funciones específicas o evitar otras, debemos incluirlo en nuestro prompt. Esto evita recibir soluciones que, aunque funcionales, no se ajusten a nuestras necesidades.

Interpretar errores en Python

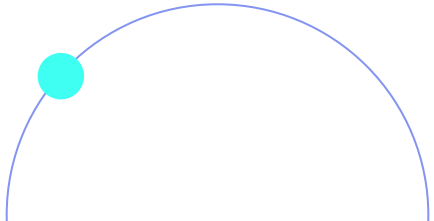
Los mensajes de error en Python **son mapas detallados** que nos guían hacia la solución.

El ***traceback***, o traza de error, nos proporciona una **secuencia cronológica inversa de los eventos que llevaron al error**, comenzando por la última línea que indica el tipo específico de error.

Veamos la imagen de las siguiente *slide*.



```
Ingrese el primer valor: 2
Ingrese el segundo valor: 3
Ingrese el tercer valor: 4
Traceback (most recent call last):
  File "c:\Users\MasterF1\Downloads\clase1.py", line 5, in <module>
    resultado = (num1 + num2 + num3) / 3
TypeError: unsupported operand type(s) for /: 'str' and 'int'
PS C:\Users\MasterF1>
```



Recuerda que los mensajes de error **no son señales de fracaso, sino herramientas de aprendizaje** que nos ayudan a mejorar nuestro código. Cada error que encontramos y resolvemos nos hace mejores programadores y nos ayuda a escribir código más robusto en el futuro.

Al comunicar estos errores a herramientas de IA, es importante incluir no solo el **mensaje de error**, sino también **el contexto que lo rodea**. Por ejemplo, si tenemos un error en una función que procesa datos, debemos mostrar la definición de la función, cómo la estamos llamando y los datos de entrada que estamos utilizando.

La IA puede entonces **analizar este contexto completo y sugerirnos soluciones específicas**.

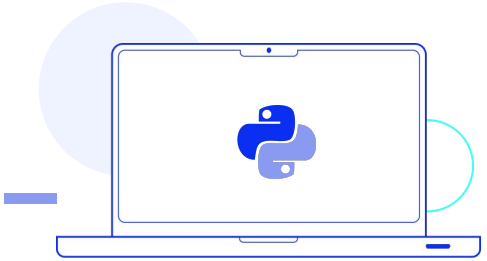


Depurador VS Code con IA

La integración de GitHub Copilot con el depurador tradicional de VS Code permite **obtener explicaciones en tiempo real mientras se ejecuta el código** paso a paso.

> Por ejemplo, cuando establecemos un punto de interrupción en una función problemática, Copilot puede analizar el estado de las variables y sugerir posibles problemas antes de que el error ocurra.

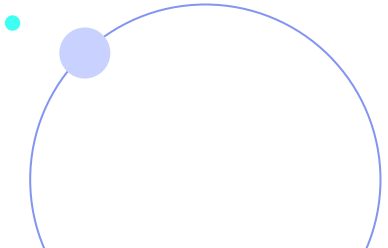
La extensión Python Intellisense, mejorada con capacidades de IA, **detecta patrones de error potenciales y sugiere mejoras** a nuestro código. Cuando escribimos código que podría generar una excepción común, la extensión nos advierte y propone alternativas más seguras.



¿Cómo desarrollar un proceso sistemático de depuración?

Antes de consultar cualquier herramienta de IA, es necesario **identificar el problema**. Para ello, debemos reproducir el error de manera consistente y documentar las condiciones exactas en que ocurre. Esto significa registrar los datos de entrada, el estado de las variables relevantes y el comportamiento esperado versus el actual.

> Por ejemplo, si tenemos un error en una aplicación que procesa datos, primero verificamos la integridad de los datos de entrada, luego el flujo de procesamiento, y finalmente las operaciones específicas que fallan. En cada paso, podemos **utilizar la IA como un consultor experto que valida nuestras hipótesis**.



La implementación de **pruebas unitarias guiadas por IA** nos ayudará a identificar casos extremos que podríamos no haber considerado. La IA puede sugerir escenarios de prueba basados en patrones comunes de error y ayudarnos a escribir aserciones más robustas.

El **diagnóstico iterativo** es esencial: comenzamos con una hipótesis sobre la causa del error, la probamos con ayuda de la IA, y refinamos nuestra comprensión basándonos en los resultados.

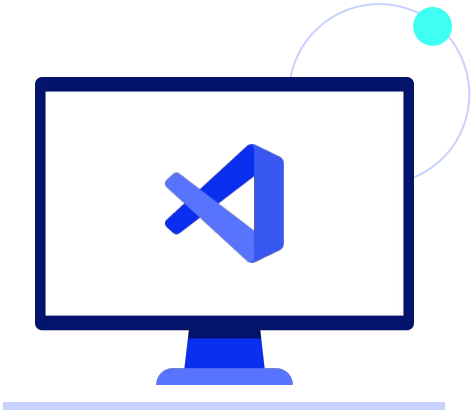
Asimismo, debemos mantener un **registro de los errores encontrados**, **las soluciones intentadas y los resultados obtenidos.** Esto nos ayuda a construir una base de conocimiento que la IA puede utilizar para sugerir soluciones más precisas en el futuro.

Finalmente, es importante mantener un equilibrio entre la automatización que nos brinda la IA y **nuestro juicio como desarrolladores**. La IA es una herramienta poderosa, pero nuestro conocimiento del contexto específico del problema y los requisitos del negocio son igualmente importantes para encontrar la solución más adecuada.

Complementos IA para VS Code

Estos son algunos complementos para Visual Studio Code que **integran inteligencia artificial para ayudar a programar, sugerir código y optimizar el flujo de trabajo**:

- GitHub Copilot
- Tabnine
- CodeWhisperer (AWS)
- AI Coding Assistant



GitHub Copilot (recomendado para Python)

- Gratis para estudiantes (a través de GitHub Education) y proyectos de código abierto.
- Costos: $10 USD/mes o $100 USD/año (con prueba gratuita de 30 días).

CodeWhisperer (AWS)

- Gratis para desarrolladores individuales.
- Planes empresariales, precio según uso.

Tabnine (recomendado para Python)

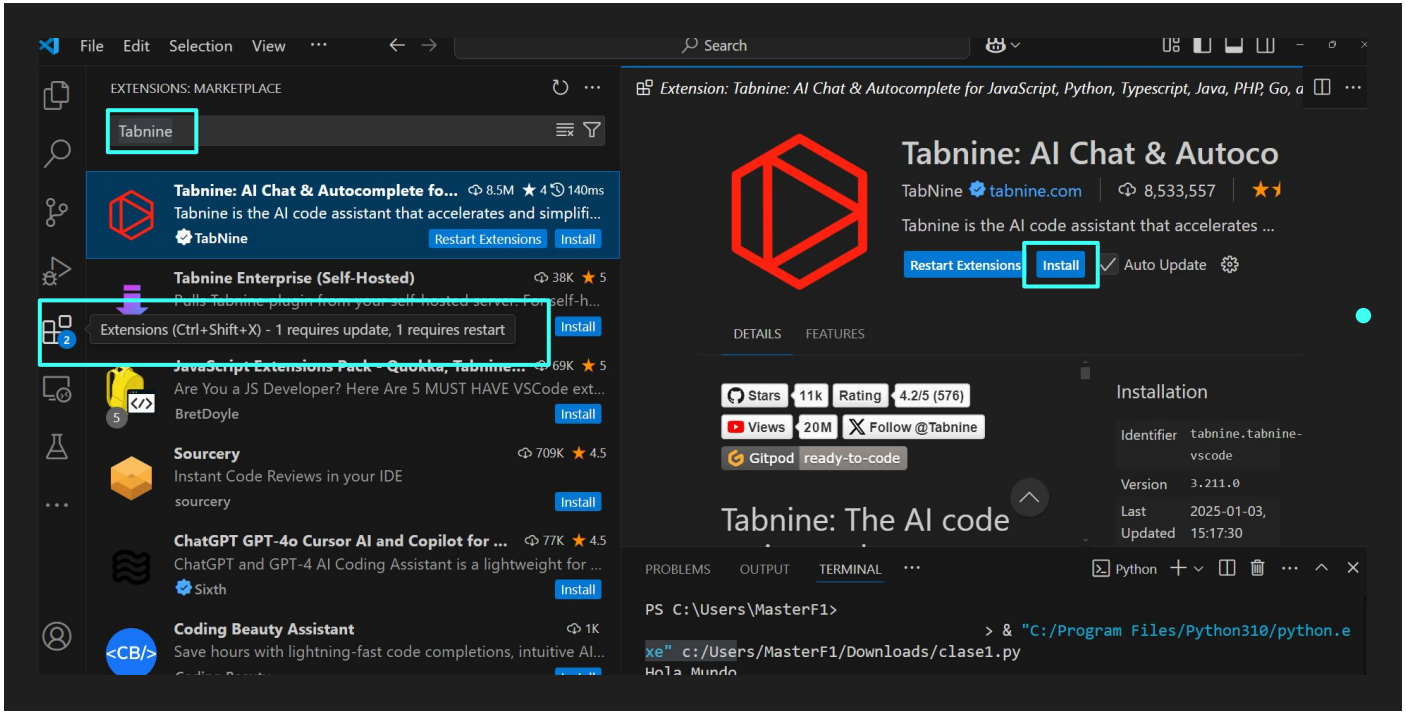
- Versión básica (gratuita) con sugerencias limitadas.
- Plan Pro: $12 USD/mes (mejor autocompletado, entrenado en tu código).
- Plan Team: $40 USD/mes/usuario (para equipos).



¿Cómo instalar un complemento?

1. Abre VS Code y ve a la barra lateral.
2. Selecciona el ícono “Extensiones” (Ctrl+Shift+X).
3. Busca el complemento deseado (por ejemplo, Tabnine).
4. Haz clic en “Instalar”.
5. Sigue las instrucciones para configurarlo.




File Edit Selection View
Search
EXTENSIONS: MARKETPLACE
Tabnine
Tabnine: AI Chat & Autocomplete fo... 8.5M 4 140ms
Tabnine is the AI code assistant that accelerates and simplifi...
TabNine
Restart Extensions
Install
Tabnine Enterprise (Self-Hosted)
38K 5
Install
Extensions (Ctrl+Shift+X) - 1 requires update, 1 requires restart
69K 5
Are You a JS Developer? Here Are 5 MUST HAVE VSCode ext...
BretDoyle
Install
Sourcery
709K 4.5
Instant Code Reviews in your IDE
sourcery
Install
ChatGPT GPT-4o Cursor AI and Copilot for ...
77K 4.5
ChatGPT and GPT-4 AI Coding Assistant is a lightweight for ...
Sixth
Install
Coding Beauty Assistant
1K
Save hours with lightning-fast code completions, intuitive AI...
Extension: Tabnine: AI Chat & Autocomplete for JavaScript, Python, Typescript, Java, PHP, Go, a
Tabnine: AI Chat & Autoco
TabNine tabnine.com
8,533,557
Tabnine is the AI code assistant that accelerates ...
Restart Extensions
Install
Auto Update
DETAILS
FEATURES
Stars 11k
Rating 4.2/5 (576)
Views 20M
Follow @Tabnine
Gitpod ready-to-code
Installation
Identifier tabnine.tabnine-vscode
Version 3.211.0
Last Updated 2025-01-03, 15:17:30
Tabnine: The AI code
PROBLEMS
OUTPUT
TERMINAL
Python
PS C:\Users\MasterF1>
> & "C:/Program Files/Python310/python.exe" c:/Users/MasterF1/Downloads/clase1.py
Hola Mundo

Ejemplo de solución de errores con Tabnine:

